

LangChain과 LangGraph 구축 실습 커리큘럼

Python 직접 구현부터 Langfuse 관찰과 평가까지

작성 및 공식 문서 확인 2026년 10월 3일 · Windows 11 · Git Bash · Docker Desktop

동일한 주문 및 결제 장애 조사 기능을 Python, LangChain, LangGraph로 구현하고 Langfuse에서 실행 근거를 확인한다. 설치와 첫 실행을 독립된 학습 목표로 두며, 프레임워크가 제공하는 동작과 직접 구현해야 하는 정책을 구분한다.



구간	회차	계획 시간
공통 준비	0-1~0-2	3~4시간
Python 직접 구현	1-1~1-4	8~10시간
LangChain 구축	2-1~2-4	8~10시간
LangGraph 구축	3-1~3-4	9~11시간
Langfuse 구축과 평가	4-1~4-4	9~12시간
종합 검증과 시연	최종 실습	3시간

총 18회와 종합 검증, 약 40~50시간의 계획이다. 코드 읽기와 환경 문제 해결에 따라 달라질 수 있다. 본 문서는 학습 및 구축 계획이며 설치 완료를 의미하지 않는다.

구축 위치와 통신 경로

로컬 구축 배치도

설치 계획 · Python 에이전트는 Windows 호스트에서, 관측 서비스는 Docker에서 실행



Python 에이전트는 우선 Windows 가상환경에서 실행한다. Docker Desktop에는 기존 관측 환경과 새 Langfuse 환경을 별도 Compose 프로젝트로 둔다. 초기에는 모델 하나를 사용하고 동일한 도구와 샘플로 구현 차이를 비교한다.

구성	접근 방식과 검증
모델 API	선택한 공급자의 인증과 HTTPS 연결을 실제 한 번 호출해 확인
관측 API	Windows 호스트에서 접근 가능한 공개 포트를 확인한 후 도구에 설정
Langfuse 서버	미사용 호스트 포트 지정. UI와 SDK 수집 요청을 각각 검증
컨테이너 내부	같은 Docker 네트워크의 서비스 이름 사용. localhost와 혼동하지 않기
영속 데이터	LangGraph 체크포인트와 Langfuse 볼륨을 구분해 재시작 검증

LangChain과 LangGraph는 애플리케이션에 설치하는 라이브러리이며 이 과정에서 별도 웹 서버가 필수는 아니다. Langfuse는 서버 환경을 구성한다. 패키지와 컨테이너는 호환성을 확인한 버전으로 고정하고 기록한다. [1][3][6]

네트워크 주소, 모델 ID, 로그 조회 API, 포트, 데이터 규모는 실제 환경 점검에서 확정한다. 현재 저장소의 파일이나 실행 상태를 확인했다고 가정하지 않는다.

공통 준비

0-1 개발 환경과 프로젝트 구성

목적 어느 Python과 어느 네트워크에서 프로그램이 실행되는지 구분한다.

개념 Windows Python, Git Bash, 가상환경, Docker 공개 포트, 환경변수와 의존성 고정.

설치와 구현 Python 실행 경로와 버전을 확인한다. agent-investigation-lab 프로젝트와 .venv를 만들고 tools, fixtures, apps, tests, docs 폴더를 준비한다. .env.example과 실제 비밀 설정을 분리하고 설치 버전을 기록한다.

실행과 예상 python -m venv .venv / source .venv/Scripts/activate / python -m pip --version. 현재 사용 포트와 Docker 자원도 점검한다. WSL Python과 Windows Python을 섞지 않는다.

실패 실험 활성 환경이 다르면 import 결과가 달라짐을 확인한다. 기존 Grafana와 신규 Langfuse의 포트 충돌 가능성을 점검한다.

완료와 증거 실행 경로, 가상환경, 포트 계획을 설명한다. 환경 점검표와 의존성 파일을 남긴다.

0-2 모델 연결과 조사 데이터 계약

목적 요청, 도구 결과, 최종 보고의 형식을 정하고 실제 모델 연결을 검증한다.

개념 모델 API, 공식 SDK, 인증, 입력과 출력 토큰, JSON 계약, 관측 시각과 조사 시간.

설치와 구현 tool calling을 지원하는 모델 하나를 선택하고 공식 SDK를 설치한다. 짧은 요청을 한 번 보내 응답과 사용량을 확인한다. 이후 공통으로 사용할 서비스 이름, 시간 범위, 증거 ID와 오류 형식을 정의한다.

실행과 예상 get_service_status, get_error_summary, get_recent_errors의 입력과 결과를 예시 JSON으로 작성한다. 정상, 지연, 연결 실패, 조회 실패 샘플을 준비한다.

실패 실험 데이터 0건과 조회 실패를 다른 상태로 표현한다. 샘플 정답 라벨은 에이전트 입력과 분리한다.

완료와 증거 사용자 요청부터 모델 응답까지 설명한다. 데이터 계약, 샘플 데이터, 실제 모델 호출 결과를 남긴다.

Python 도구와 모델 연결

1-1 Python 조회 도구 구현

목적 모델 없이도 도구가 독립적으로 실행되고 결과를 검증할 수 있게 만든다.

개념 함수, 딕셔너리, JSON 읽기, 입력 검증, 예외 처리, 성공과 오류의 구분.

설치와 구현 샘플 파일을 읽는 상태 조회 함수를 작성한다. 서비스와 시간 범위를 검증하고 출처, 관측 시각, 성공 여부, 증거를 구조화해 반환한다.

실행과 예상 정상 서비스, 지연 서비스, 존재하지 않는 서비스의 결과를 직접 호출해 비교한다. 호출 전 예상 JSON을 작성한다.

실패 실험 누락된 필드, 잘못된 서비스, 자료 없음 조건을 주입한다. 도구 실패를 정상 결과로 숨기지 않는다.

완료와 증거 함수의 입력과 반환값을 직접 설명한다. tools 코드와 세 종류 이상의 입출력 증거를 남긴다.

1-2 모델의 도구 요청을 실제 실행으로 연결

목적 모델이 선택하고 프로그램이 실행한다는 경계를 코드로 확인한다.

개념 도구 이름, 설명, 입력 스키마, tool call ID, 실행 허용 목록, 결과 메시지.

설치와 구현 모델에 도구 정의를 전달한다. 도구 호출 응답을 받아 허용된 함수와 인자를 검증하고 실행한다. 원래 호출 ID에 맞춰 결과를 돌려주고 최종 설명을 받는다.

실행과 예상 일반 설명 질문과 실제 조회 질문을 비교한다. 우선 한 개의 도구로 요청, 선택, 실행, 결과 전달을 순서대로 출력한다.

실패 실험 지원하지 않는 도구와 잘못된 인자는 재현용 응답으로 검사한다. 모델이 항상 오류를 만들 것이라고 가정하지 않는다.

완료와 증거 실제 API 실행 코드와 모델 응답을 구분한다. 도구 호출 요청, 함수 결과, 최종 답변을 한 실행 ID로 묶는다.

Python 반복과 실제 환경 연결

1-3 에이전트 반복과 종료 구현

목적 도구 하나의 실행을 여러 번의 조사로 확장하고 실행을 제한한다.

개념 메시지 이력, 반복문, 조사 상태, 모델 호출과 도구 호출의 차이, 종료 사유.

설치와 구현 상태 조회와 오류 조회를 연결한다. 모델이 추가 도구를 요청하면 반복하고, 답변 또는 횟수·시간 제한에 도달하면 종료한다. 현재 증거와 미확인 사항을 부분 보고로 남긴다.

실행과 예상 처음에는 충분한 증거, 다음에는 추가 조회가 필요한 증거를 제공한다. 모델 호출 수와 도구 실행 수를 각각 기록한다.

실패 실험 같은 조회 반복, 여러 도구 요청, 무응답, 시간 제한을 시험한다. SDK 통신 재시도와 에이전트 재조사를 구분한다.

완료와 증거 정상 완료, 제한 종료, 오류 종료를 설명한다. 무한 반복 없이 종료한 로그와 제한값의 근거를 남긴다.

1-4 실제 관측 데이터에 연결

목적 샘플 기반 구조가 실제 조회에서도 올바르게 동작하는지 확인한다.

개념 호스트와 컨테이너 주소, 조회 시간 구간, API timeout, 메트릭 수집 상태와 서비스 상태.

설치와 구현 기존 관측 환경의 실제 포트, 메트릭, 로그 검색 경로를 읽고 도구 하나를 실제 API로 교체한다. 사람이 직접 실행한 조회와 도구 결과, 에이전트 해석을 순서대로 대조한다.

실행과 예상 현재 서비스 상태와 과거 장애 구간을 별도로 조회한다. 실제 로그 연동은 수집 경로가 확인된 경우에 추가한다.

실패 실험 빈 결과를 정상으로 해석하지 않는다. up=0은 해당 시점의 수집 실패 신호이며 프로세스 종료의 단독 증거로 사용하지 않는다.

완료와 증거 인용한 수치와 원본 데이터가 일치한다. 실제 연동 범위와 샘플로만 검증한 범위를 별도 기록한다.

LangChain 설치와 에이전트 구축

2-1 LangChain 설치와 모델 연결

목적 LangChain 본체와 모델 공급자 패키지의 역할을 구분하고 첫 호출을 실행한다.

개념 langchain, 공급자 연동 패키지, 모델 객체, 메시지, invoke, 모델 설정.

설치와 구현 기존 가상환경에 langchain과 선택한 공급자 패키지를 설치한다. 설치 목록을 기록하고 모델 객체를 만든다. 0-2와 같은 입력을 보내 공식 SDK 직접 호출과 비교한다.

실행과 예상 설치 예: `python -m pip install langchain langchain-anthropic`. 공급자 패키지는 선택 모델에 맞게 바꾸며, 실제 사용 버전은 검증 뒤 고정한다.

실패 실험 import 오류, 인증 오류, 모델 ID 오류를 구분한다. LangChain 설치만으로 모델 가중치나 별도 모델 서버가 설치되는 것은 아님을 설명한다.

완료와 증거 패키지 import와 실제 모델 호출을 각각 확인한다. 버전 기록, 모델 설정, 최초 실행 증거를 남긴다.

2-2 LangChain 에이전트 구축

목적 create_agent로 같은 도구를 실행하고 기본 에이전트 반복 구조를 관찰한다.

개념 도구 등록, create_agent, 시스템 지침, 사용자 메시지, ToolMessage와 호출 연결.

설치와 구현 1단계의 조회 함수를 공통 도구로 재사용한다. 우선 상태 조회 하나로 에이전트를 만들고, 이후 오류와 지연 조회를 추가한다. 샘플부터 같은 요청을 실행한다.

실행과 예상 도구가 필요한 요청과 필요 없는 요청을 비교한다. 이벤트나 메시지 이력으로 실제 호출한 도구, 인자, 결과를 확인한다.

실패 실험 최종 문장만 보고 성공 판정하지 않는다. 필요한 증거를 실제로 조회했는지와 제공받은 값이 답변에 보존됐는지 검사한다.

완료와 증거 LangChain으로 실행되는 장애 조사 에이전트를 시연한다. 직접 작성한 반복문 중 프레임워크가 대신한 부분을 설명한다.

LangChain 실행 정책과 구현 비교

2-3 LangChain 정책과 실패 처리

목적 모델 지침과 코드로 강제하는 실행 정책을 구분한다.

개념 미들웨어 또는 실행 래퍼, 인자 검증, 호출 제한, 도구 오류 반환, 실행시간 제한.

설치와 구현 선택한 버전의 공식 인터페이스에 맞춰 호출 제한과 도구 검증을 구현한다. 프롬프트에는 조사 원칙을 적고, 실행 코드에는 허용 도구·시간·횟수 제한을 둔다.

실행과 예상 잘못된 서비스 요청, 조회 실패, 반복 호출 상황을 실행한다. 정책 적용 전후에 어떤 호출이 차단되고 어떤 오류가 모델에 전달되는지 비교한다.

실패 실험 프롬프트의 최대 두 번 같은 문장만으로 제한이 보장된다고 가정하지 않는다. 오류 종류별로 재시도할지 종료할지 정한다.

완료와 증거 실패에도 프로그램이 제한 안에서 끝난다. 정책 코드, 오류 분류표와 종료 증거를 남긴다.

2-4 Python과 LangChain 구현 비교

목적 두 구현의 품질과 제어 방식을 같은 입력으로 비교한다.

개념 동일 조건 비교, 모델 설정, 고정 샘플, 실행 기록, 기능과 표현의 차이.

설치와 구현 공통 샘플 4개를 같은 모델·도구·정책으로 실행한다. 사용한 코드, 도구 호출, 종료 사유, 수집한 증거를 비교한다. 모델 문장의 완전 일치를 요구하지 않는다.

실행과 예상 직접 제어가 쉬운 부분과 제공 구조로 구현량이 줄어든 부분을 코드 위치로 기록한다. 이후 LangGraph로 옮길 조사 절차를 정한다.

실패 실험 프레임워크 교체와 프롬프트 변경을 한꺼번에 하지 않는다. 서로 다른 실제 시간 데이터로 품질 차이를 단정하지 않는다.

완료와 증거 LangGraph를 직접 사용할 이유를 설명한다. 비교표와 다음 단계에서 필요한 분기·상태 설계안을 남긴다.

LangGraph 설치와 조사 흐름 구축

3-1 LangGraph 설치와 첫 그래프

목적 설치 확인과 실제 그래프 실행을 구분하고 State·Node·Edge를 익힌다.

개념 StateGraph, START, END, 노드, 연결, compile, invoke, 상태 업데이트.

설치와 구현 langgraph를 명시적으로 확인·설치하고 버전을 기록한다. LangChain 의존성으로 이미 있어도 직접 사용 가능한지 검사한다. 모델 없는 두 노드 그래프로 상태 전달을 실행한다.

실행과 예상 설치 예: `python -m pip install langgraph`. 입력 검증 노드와 결과 생성 노드를 연결하고 각 단계 전후 상태를 출력한다.

실패 실험 메시지를 덮어쓰는 경우와 누적하는 경우를 비교한다. import 성공과 그래프 실행 성공을 별도 검증한다.

완료와 증거 노드와 연결을 한 개씩 수정하고 실행 경로를 예측한다. 첫 그래프 코드, 그림, 상태 변화 기록을 남긴다.

3-2 장애 조사 그래프 구축

목적 기본 확인은 코드로 고정하고 추가 조사는 모델에게 맡기는 흐름을 설계한다.

개념 모델 노드, 도구 노드, 조건부 연결, 증거 상태, 고정 절차와 모델 판단.

설치와 구현 기존 모델과 공통 도구를 그래프에 배치한다. 입력과 조사 범위를 검증하고, 모델 응답이 도구 요청이면 실행으로, 최종 답변이면 보고 검증으로 분기한다.

실행과 예상 기본 증거로 충분한 경우, 추가 조회가 필요한 경우, 자료가 부족한 경우를 비교한다. 정책상 필수 조회가 있으면 모델과 별도 노드로 명시한다.

실패 실험 최종 답변이 나왔다고 내용이 맞다고 가정하지 않는다. 사실, 원인 후보, 미확인 사항과 증거 ID를 검증한다.

완료와 증거 어느 분기를 코드가 정하고 어느 선택을 모델이 하는지 설명한다. 시나리오별 경로와 상태를 남긴다.

LangGraph 오류 처리와 상태 복원

3-3 LangGraph 오류와 실행 예산 제어

목적 에러 경로와 제한 종료를 명시적으로 구현한다.

개념 노드 오류, 재시도 정책, 실행 예산, 제한 종료, 부분 보고, 상태 보존.

설치와 구현 모델 호출과 도구 실행 전에 예산을 확인한다. 일시 통신 실패는 제한적으로 재시도하고 잘못된 입력은 무의미하게 반복하지 않는다. 제한 소진 시 추가 모델 호출 없이 부분 보고를 만들 수 있게 한다.

실행과 예상 정상, 일시 실패, 지속 실패, 반복 요청을 비교한다. 그래프 단계 제한과 실제 모델·도구 호출 예산은 별도로 기록한다.

실패 실험 `recursion_limit`을 실제 API 호출 횟수와 동일시하지 않는다. 하위 SDK의 자동 재시도도 사용량과 시간에 영향을 줄 수 있다.

완료와 증거 모든 실패 시나리오가 정해진 경로로 종료된다. 제한값, 종료 사유, 남은 증거를 추적할 수 있다.

3-4 체크포인트와 중단 재개 구축

목적 조사 상태를 저장하고 이어가는 기능을 실제로 검증한다.

개념 `thread_id`, 체크포인트, 메모리와 SQLite, `interrupt`와 `resume`, 재실행과 자료 신선도.

설치와 구현 처음에는 메모리 체크포인트로 같은 작업을 이어간다. 다음으로 SQLite용 체크포인트 패키지를 연결한다. 조사 중 추가 시간 범위를 입력받도록 중단하고 재개한다.

실행과 예상 같은 `thread`와 새 `thread`를 비교하고 프로세스 재시작도 시험한다. 저장된 증거의 시각을 보고 재조회가 필요한지 판단한다.

실패 실험 메모리 저장은 프로세스 재시작을 견디지 못한다. 재개 과정에서 노드 코드가 다시 실행돼 조회가 반복되는지 확인한다.

완료와 증거 저장 위치, 작업 식별자, 재개 경로를 설명한다. 체크포인트, 재시작 전후 결과와 중복 실행 확인 기록을 남긴다.

Langfuse 서버 구축과 추적 연동

4-1 Langfuse 서버 구축

목적 Docker Compose로 관측 서버를 직접 구성하고 데이터 지속성을 확인한다.

개념 Web와 Worker, 데이터 저장소, 환경변수, 포트 매핑, 볼륨, 프로젝트와 API 키.

설치와 구현 공식 Compose의 선택 버전과 의존 서비스를 검토해 별도 프로젝트로 구성한다. 기존 포트 충돌을 피하고 로컬 비밀 설정을 적용한다. 컨테이너 상태와 로그를 확인한 뒤 사용자·프로젝트·연동 키를 만든다.

실행과 예상 Langfuse 공개 포트는 미사용 값으로 정한다. 예를 들어 3300은 계획값일 뿐이며 실제 사용 여부를 확인한다. 컨테이너 재생성 후 프로젝트 데이터가 유지되는지 검증한다.

실패 실험 UI 접속 성공만으로 수집 성공을 판단하지 않는다. 의존 서비스 실패, 볼륨 미연결, 호스트와 컨테이너 URL 혼동을 점검한다.

완료와 증거 UI, 프로젝트, 연동 키, 영속 볼륨을 확인한다. Compose와 설정 예시, 구성도, 기동 및 재기동 결과를 남긴다.

4-2 Python과 LangChain 및 LangGraph 추적 연동

목적 세 구현의 실행을 같은 기준으로 관찰한다.

개념 Trace, 관측 단위, 부모와 자식 관계, 콜백, 명시적 계측, 전송과 flush.

설치와 구현 langfuse SDK를 설치하고 최소 이벤트를 보낸다. Python 직접 구현에는 명시적으로 계측하고, LangChain·LangGraph에는 콜백과 필요한 사용자 정의 계측을 추가한다.

실행과 예상 입력, 모델 호출, 도구 결과, 분기, 최종 보고를 한 조사 Trace로 연결한다. 프로세스 종료 전 기록이 전송되는지도 확인한다.

실패 실험 콜백과 수동 계측으로 같은 모델 호출이 중복 기록되지 않는지 본다. 서비스 요청 Trace와 에이전트 조사 Trace는 참조 ID로 구분한다.

완료와 증거 원본 실행 로그와 Trace의 호출 횟수 및 결과가 맞는다. 부모·자식 구조, 입력·출력·시간을 화면에서 설명한다.

Langfuse 평가와 개선

4-3 실행 경로와 답변 품질 평가

목적 정답뿐 아니라 증거 수집과 종료 동작을 평가한다.

개념 고정 데이터셋, 기대 행동, 금지 결론, 코드 검사, 사람 평가, 보조 모델 평가.

설치와 구현 정상, 지연, 연결 거부, 조회 실패, 자료 없음, 시간 불일치, 증거 충돌, 반복 요청의 8개 사례를 만든다. 출력 형식과 호출 제한은 코드로 검사하고 증거와 결론 연결은 사람이 검토한다.

실행과 예상 먼저 각 사례 1회로 구조 오류를 찾는다. 수정 후 같은 샘플을 여러 번 실행해 변동을 확인한다. 실제 연동 결과는 별도 세트로 분리한다.

실패 실험 평가 정답을 에이전트 입력에 섞지 않는다. 모델 평가 점수만으로 합격을 확정하지 않고 실패 사례를 직접 읽는다.

완료와 증거 시나리오별 통과 여부, 실패 이유, 수정 후보를 제시한다. 데이터셋과 평가표, 대표 Trace를 남긴다.

4-4 품질과 시간 및 비용 개선

목적 하나의 변경이 결과에 미치는 영향을 비교하고 한계를 보고한다.

개념 입력·출력 토큰, 추정 비용, 전체 경과시간, 중복 조회, 비교 조건, 소표본 한계.

설치와 구현 모델과 고정 데이터셋을 유지하고 도구 설명 또는 중복 증거 처리 중 하나만 바꾼다. 통과 건수, 근거 없는 단정, 호출 횟수, 시간, 토큰과 비용을 비교한다.

실행과 예상 느린 도구, 재시도, 불필요한 반복 사례의 시간 구간을 분석한다. Langfuse 비용 계산에 필요한 모델 매핑·사용량·가격을 확인하고 공급자 사용 기록과 대조한다.

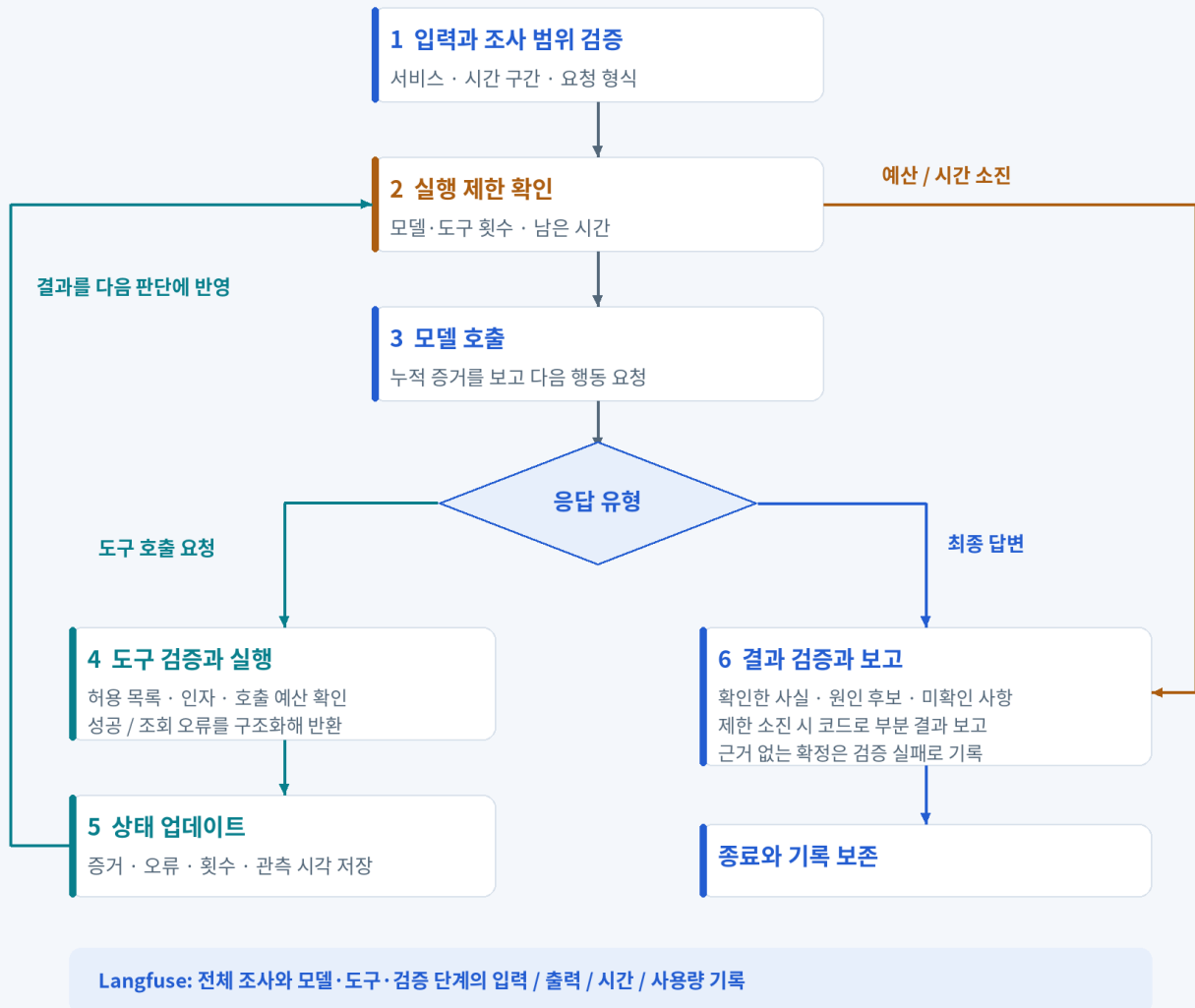
실패 실험 필요한 맥락을 줄여 품질이 떨어진 경우 개선으로 판정하지 않는다. 적은 표본의 p95나 운영 처리량을 일반화하지 않는다.

완료와 증거 개선 전후 결과와 남은 한계를 설명한다. 최종 보고서, 대표 Trace, 3분 시연 대본과 재현 절차를 남긴다.

에이전트 실행 워크플로우

장애 조사 에이전트 실행 워크플로우

LangGraph에서 직접 설계할 흐름 · 모델의 판단과 프로그램의 실행을 구분합니다



모델은 다음 행동을 요청하고 실행 코드는 도구와 인자, 예산을 검증한다. 성공한 조회와 조회 오류는 다른 상태로 보관하며, 누적 결과를 다음 판단에 전달한다. 최종 보고의 구조와 근거를 검사하되, 형식 검사가 내용의 진실성을 보장하지는 않는다.

제한 소진 시에는 추가 유료 호출 없이 코드로 현재 증거와 종료 사유를 보고할 수 있게 한다. 에이전트의 실행 기록은 모델 내부 사고 전체가 아니라 계획한 입력, 출력, 호출과 상태 변화다. [4][7][8]

평가 시나리오와 완료 기준

시나리오	기대 행동	피해야 할 결론
지정 구간 정상	관측 범위 내 이상 증거 없음 보고	전체 시스템 정상 확정
결제 지연	지연과 주문 실패 관계 설명	프로세스 종료 단정
연결 거부	연결 실패 증거와 원인 후보 구분	timeout과 동일시
조회 도구 실패	증거 확보 실패와 한계 보고	서비스 장애로 단정
자료 없음	자료 부족과 추가 확인 제시	오류가 없었다고 확정
시간 불일치	시간 범위를 맞춰 재확인	무관한 사건을 연결
증거 충돌	모순을 명시하고 판단 보류	편한 증거만 선택
반복 호출	정해진 제한 안에서 종료	끝없는 재조사

종합 검증

실습자가 장애 조건을 설정하되 정답 이름은 에이전트에 전달하지 않는다. 요청, 수집 증거, 그래프 경로, Langfuse Trace, 최종 보고를 차례로 확인한다. 샘플 기반 평가와 실제 환경 검증을 구분하고 같은 고정 입력으로 구현 간 차이를 비교한다.

설치 게이트 패키지 import와 모델 응답, 그래프 최초 실행, Langfuse UI와 수집 성공을 각각 확인한다.

기능 게이트 필요한 도구를 실제 실행하고 입력과 결과가 연결돼야 한다. 제한 종료와 조회 오류가 명시적으로 처리돼야 한다.

평가 게이트 오류가 있었는지뿐 아니라 실패 유형과 수정할 부분을 설명한다. 미확인 범위와 아직 검증하지 않은 동작을 보고서에 남긴다.

최종 시연은 요청 입력 → 도구 결과 → 그래프 경로 → Trace 분석 → 정상과 실패 사례 비교 순서로 구성한다. 문장이 같다는 것보다 증거, 정책 준수, 결론의 타당성을 확인한다.

진행 방식과 산출물

매 회차 진행

개념 설명 → 코드와 설정 해석 → 실행 결과 예측 → 작은 단위 실행 → 결과 분석 → 증거 기록 순서로 진행한다. 한 회차를 마칠 때 코드를 보고 동작 이유를 설명할 수 있어야 한다. 여러 단계 설치와 실행을 하나의 자동화 명령으로 숨기지 않는다.

위치 예시	작성할 내용
tools/	세 구현에서 공통 사용: 상태, 메트릭, 오류 조회와 입력 검증
fixtures/	고정 입력 샘플. 평가 정답은 모델 입력과 분리
apps/python_agent.py	직접 작성한 모델 호출과 도구 실행 반복
apps/langchain_agent.py	LangChain create_agent 기반 구현
apps/langgraph_agent.py	직접 정의한 State, Node, Edge, 오류 및 재개 경로
infra/langfuse/	선택한 공식 Compose 기반 구성과 비밀값 없는 설정 예시
tests/	데이터 계약, 실행 제한, 실패 경로와 평가 시나리오
docs/evidence/	입출력, 실행 ID, 설정과 버전, Trace 참조, 결과 해석
docs/final-report	개선 전후 비교, 검증 범위, 남은 한계, 재현과 시연 절차

기록 양식

목적: 무엇을 확인하려 했는가. 예상: 실행 전 어떤 결과를 예상했는가. 변경: 무엇을 바꿨는가. 관측: 실제 무엇이 나왔는가. 판단: 증거로 어디까지 말할 수 있는가. 다음 단계: 무엇이 아직 남았는가.

선택 확장

MCP로 조회 도구 공개, 운영 문서 검색, 장기 메모리, 여러 에이전트 비교, 클라우드 배포, 동시 조사 부하 테스트는 본 과정 완료 후 목적에 따라 선택한다. 기본 과정의 완료를 위해 모두 구축할 필요는 없다.

공식 참고 문서와 실행 전 확인사항

[1] LangChain 설치

<https://docs.langchain.com/oss/python/langchain/install>

[2] LangChain 에이전트 개요

<https://docs.langchain.com/oss/python/langchain/overview>

[3] LangGraph 설치

<https://docs.langchain.com/oss/python/langgraph/install>

[4] LangGraph 첫 그래프와 도구 호출

<https://docs.langchain.com/oss/python/langgraph/quickstart>

[5] LangGraph 상태 저장

<https://docs.langchain.com/oss/python/langgraph/persistence>

[6] Langfuse Docker Compose 구축

<https://langfuse.com/self-hosting/deployment/docker-compose>

[7] Langfuse 추적과 관측

<https://langfuse.com/docs/observability/overview>

[8] LangChain 및 LangGraph 추적 연동

<https://langfuse.com/integrations/frameworks/langchain>

[9] Langfuse 평가

<https://langfuse.com/docs/evaluation/overview>

[10] Langfuse 토큰과 비용

<https://langfuse.com/docs/observability/features/token-and-cost-tracking>

[11] LangGraph 중단과 재개

<https://docs.langchain.com/oss/python/langgraph/interrupts>

설치 전 확인

공식 문서는 변경되므로 실습 시작 시 선택한 Python, SDK, LangChain, LangGraph, Langfuse 서버 및 SDK의 호환 버전을 확인한다. LangChain 에이전트는 내부적으로 LangGraph를 사용하므로 신규 설치 여부와 직접 그래프를 구현하는 경험은 구분한다. [1][2][3]

여기서 pip 명령은 설치 대상의 역할을 보여준다. 실제 실행 시 선택한 모델 공급자 패키지를 사용하고 검증한 버전을 의존성 파일에 고정한다. 환경 점검 전 모든 명령을 한꺼번에 실행하는 절차가 아니다.